

Complete Guide to Docker Volumes and Persistent Data

By default, Docker containers lose all saved data the moment they stop or get recreated. Docker volumes fix this problem by saving your databases, logs, and files safely on your host computer.

What you'll learn:

1. Named Volumes for Persistent Storage
2. Bind Mounts for Live Development
3. Inspecting Volume Locations
4. Cleaning Up Orphaned Storage

Aman Raj Singh

Cloud Operations Engineer @ iManage

linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

1. Named Volumes for Persistent Storage

Named volumes are the best choice for production databases and permanent storage. Docker creates a folder in a managed system path and controls its lifecycle. Even if you delete or update the app container, the volume remains completely intact and untouched. You can easily attach the same volume to a brand-new container later.

```
$# Create a named volume
docker volume create my_app_data

# Attach it to a Postgres container
docker run -d --name postgres-db -v my_app_data:/var/lib/postgresql/data postgres
```

Cloud & DevOps Daily

Docker Volumes Explained: Where Does Your Data Go?

Sunday, August 23, 2026

Swipe to tip 2 >> linkedin.com/in/mramanrajsingh

2

TIP 2 OF 4

2. Bind Mounts for Live Development

Bind mounts map a specific directory from your host laptop directly into the container. Any edit you make in your local code editor instantly reflects inside the running container without rebuilding. This is ideal for active local development and hot-reloading web frameworks.

```
$# Mount your current local directory into /app inside the container
docker run -d -p 8080:8080 -v $(pwd):/app node:18-alpine npm start
```

Cloud & DevOps Daily

Docker Volumes Explained: Where Does Your Data Go?

Sunday, August 23, 2026

Swipe to tip 3 >> linkedin.com/in/mramanrajsingh

3

TIP 3 OF 4

3. Inspecting Volume Locations

Sometimes you need to know where Docker is physically storing your volume files on the host system. The inspect command returns JSON output containing metadata, including the absolute path under the Mountpoint field. This helps you verify permissions and back up raw files directly.

```
$# Inspect detailed configuration of a volume
docker volume inspect my_app_data
```

Cloud & DevOps Daily

Docker Volumes Explained: Where Does Your Data Go?

Sunday, August 23, 2026

Swipe to tip 4 >> linkedin.com/in/mramanrajsingh

4

TIP 4 OF 4

4. Cleaning Up Orphaned Storage

When you delete containers, Docker keeps their associated volumes to prevent accidental data loss. Over time, dozens of unused volumes can accumulate and eat up gigabytes of disk space. Running the prune command removes all unattached volumes in one safe action.

```
$# Safely delete all volumes not currently used by any container  
docker volume prune -f
```


Cloud & DevOps Daily

Docker Volumes Explained: Where Does Your Data Go?

Sunday, August 23, 2026

See Key Takeaways on next page >> linkedin.com/in/mrmanrajsingh

Key Takeaways

- + Always use named volumes for databases instead of anonymous volumes.
- + Use bind mounts strictly during local development and testing.
- + Set read-only permissions (:ro) on mounts when containers only need to read data.
- + Keep volume names descriptive according to the project and environment.
- + Regularly inspect and prune unused volumes to free up disk space.

Watch Out For

- ! Assuming 'docker rm' deletes container volumes (it leaves them behind).
- ! Using bind mounts in production, which causes host-dependency issues.
- ! Overwriting container application files by mounting over non-empty directories.

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mrmanrajsingh